

MSCS & MSDS OOP with Python Assignment 2

Advent 2026

Instructions

Each question below is a **mini-project**: an open problem set in a Ugandan context. You are not just computing answers. You are building a small, reusable, well-tested piece of software and interpreting what it tells you. Where a task is open-ended, state your assumptions and justify your choices.

- Submit **one GitHub repository link** on Moodle. Use one Jupyter Notebook per mini-project (e.g. `project1_population.ipynb`). Put reusable logic in `.py` modules under `src/` and import it into the notebooks.
- The repository must include a `README.md` (how to set it up and run it, plus a summary of findings), a `requirements.txt` or `environment.yml`, and a `tests/` folder.
- **OOP is required.** Each mini-project must model its domain with at least one well-designed class: attributes, methods, docstrings and type hints. Use inheritance, composition or dunder methods where they genuinely help.
- **Reproducibility:** fix random seeds (`np.random.default_rng(seed)`). A notebook must run top to bottom with *Restart & Run All* without errors.
- **Testing:** write at least **three** `pytest` unit tests per mini-project, covering normal cases and at least one edge case (empty input, zero, invalid value).
- **Written analysis:** end every mini-project with a *Findings & Limitations* section (150-300 words) that interprets the results, not just restates them.
- Data marked *illustrative* is synthetic. You may replace it with real data from a cited source (UBOS, UNMA, NASA POWER, Bank of Uganda, etc.) for extra credit.
- Libraries expected: `numpy`, `scipy`, `statistics`, `matplotlib` (plus `pandas` if you wish). Avoid black-box forecasting libraries unless a task says otherwise. The point is to implement the methods yourself.

Academic integrity & AI use: You may use AI coding assistants, but you must declare how you used them in your README. You may be asked to explain any line of your code in a short viva. Identical repositories will be investigated.

Mini-Project 1: UBOS District Population Forecaster

Scenario. A district planning unit wants 5-year population forecasts to plan primary school classrooms. You are given 10 years of population estimates (in thousands, *illustrative*, 2015-2024):

```
Kampala = [1200, 1250, 1300, 1350, 1420, 1500, 1580, 1650, 1720, 1800]
Wakiso  = [ 950, 1000, 1070, 1150, 1220, 1300, 1390, 1480, 1570, 1670]
Gulu    = [ 320, 330, 345, 360, 375, 390, 410, 430, 455, 480]
```

Core tasks

1. Design a `DistrictPopulation` class that stores years and populations as NumPy arrays, validates the input (equal lengths, no negative values) and implements `__repr__` and `__len__`. Add **two more districts** of your choice.
2. Compute the mean, median, variance and standard deviation for each district with the `statistics` module, then again with NumPy. Explain why `statistics.variance` and `np.var` give different numbers, and what the `ddof` argument does.
3. Compute year-on-year growth rates and the Compound Annual Growth Rate (CAGR) for each district. Which district is growing fastest in relative terms?
4. Implement **three forecasting models** as subclasses of an abstract `Forecaster` base class that exposes `fit()` and `predict(horizon)`: (a) linear trend with `np.polyfit`, (b) exponential/CAGR growth, and (c) a *Fibonacci-ratio* model that scales the last value by successive Fibonacci ratios (the approach from the previous cohort's assignment).
5. **Validate before you forecast:** train each model on 2015-2021 and test it on 2022-2024. Report MAE, RMSE and MAPE in one comparison table per district, and select the best model for each district.
6. Use the selected model to forecast 2025-2029. Compare the variance of the actual series against the variance of the forecast series, and explain what the difference tells you.
7. Plot actual, fitted and forecast values for every district on a single figure made of subplots, with the train/test split clearly marked.
8. **Planning output:** assume 18% of the population is of primary-school age and a classroom holds 53 pupils. Estimate how many *additional* classrooms each district needs by 2029.

Extension

- Use bootstrap resampling of residuals (at least 1,000 resamples) to add 95% prediction intervals to your forecasts. Shade them on the plot.
- Critically assess the Fibonacci-ratio model. Under what growth assumptions (if any) is it defensible?

Mini-Project 2: Solar Micro-Grid Dispatch Planner

Scenario. A rural health centre in Kasese runs a micro-grid. Each day, the energy drawn from solar panels (x) and batteries (y) must satisfy two demand constraints:

$$\begin{aligned} 3x + 2y &= D1 && \text{(daytime load, kWh)} \\ 4x + y &= D2 && \text{(critical-equipment load, kWh)} \end{aligned}$$

Core tasks

1. Create a `MicroGrid` class that holds the coefficient matrix and exposes `solve_day(d1, d2)`. Before solving, check that the system is well-posed: compute the determinant and the condition number (`np.linalg.cond`) and explain what they mean for this system.
2. Support two input modes: (a) interactive `input()` with robust validation (reject non-numeric, negative and empty values, and re-prompt), and (b) loading **30 days** of demand from a CSV file. Generate the CSV yourself with a seeded random generator: realistic values with a weekly pattern and some noise.
3. Solve all 30 days with `scipy.linalg.solve`, first in a Python loop and then in a single vectorised call (pass a 2×30 right-hand side). Time both approaches with `timeit` and comment on the difference.
4. Detect and flag **physically infeasible days** (negative x or y). Propose and implement a sensible handling strategy, for example clipping plus a report, or switching to a non-negative least-squares solver (`scipy.optimize.nnls`).
5. Use `statistics` to report the mean, variance and standard deviation of solar and battery usage. Which source is more volatile?
6. **Cost model:** given solar at UGX 150/kWh and battery at UGX 450/kWh (*illustrative*), compute the daily and monthly energy cost.
7. Plot daily solar vs battery usage as a stacked bar chart, with a second axis showing daily cost.

Extension

- Add a diesel generator (z) and a third constraint of your choice. Build a `HybridMicroGrid (MicroGrid)` subclass that solves the 3×3 system, and discuss what happens if the new equation is linearly dependent on the others.
- **Sensitivity analysis:** perturb $D1$ and $D2$ by $\pm 5\%$ (Monte Carlo, 1,000 draws) and show how much x and y move. Relate the result to the condition number.

Mini-Project 3: Lake Victoria Fish Stock & Export Risk Model

Scenario. A fish-export cooperative in Jinja wants to know (a) whether its harvesting rate is sustainable and (b) how risky its revenue is. The previous version of this question modelled fish stock with Fibonacci numbers and flagged risk when revenue variance exceeded 50,000. You will build a more defensible model and explain why the old approach is flawed.

Core tasks

1. As a **baseline**, generate 15 Fibonacci numbers as "stock" and explain in 3-5 sentences why unbounded Fibonacci growth is biologically unrealistic.
2. Implement a **FishStock** class using the discrete **logistic growth model with harvesting**: $N(t+1) = N(t) + r \cdot N(t) \cdot (1 - N(t)/K) - h \cdot N(t)$. Use $r = 0.4$, $K = 10,000$ tonnes, $N(0) = 4,000$ tonnes and simulate 52 weeks.
3. Implement a **PriceModel** class that simulates the weekly price in UGX/kg as a seeded random walk starting at 12,000, with bounds (e.g. 9,000-16,000). Weekly revenue = harvest (kg) $\times$ price.
4. Compute the mean, median, variance, standard deviation and **coefficient of variation (CV)** of revenue with **statistics**. Explain why a raw variance threshold such as 50,000 is meaningless here. (Hint: what are the units of variance?)
5. Build a **RiskAssessor** class that classifies risk with a justified rule based on CV. Run a **Monte Carlo simulation** ($\geq 1,000$ price paths) and report the 5% **Value-at-Risk** of annual revenue.
6. **Scenario analysis**: run harvest rates $h = 0.05, 0.10, 0.20$ and 0.30 . For each one, report the final stock, total revenue and risk class. Compare your results with the theoretical maximum sustainable yield, $MSY = rK/4$.
7. Produce (a) stock trajectories for each harvest rate on one chart, and (b) a histogram of simulated annual revenue with the VaR marked.

Extension

- Add a closed season (no harvesting for 8 weeks per year) and quantify its effect on long-run revenue over 5 years.

Mini-Project 4: Rainfall Pattern & Crop Suitability Analyser

Scenario. An agricultural extension officer wants to compare rainfall regimes across regions and advise farmers on which months suit which crops. Monthly rainfall in mm, Jan-Dec (*illustrative*):

```
Kampala = [120, 140, 180, 200, 220, 180, 90, 70, 60, 100, 110, 130]
Gulu    = [ 8, 25, 75, 160, 190, 145, 170, 215, 175, 150, 60, 15]
Mbarara = [70, 85, 120, 140, 90, 25, 20, 55, 100, 125, 120, 90]
```

Core tasks

1. Create a **Region** class that holds its rainfall as a NumPy array and has methods for annual total, mean, wettest/driest month, and coefficient of variation.
2. Create a **CropRule** class that defines a suitable monthly rainfall range. Define at least **three crops** (e.g. maize, beans, coffee), with thresholds justified by a cited source. Classify every month in every region, e.g. "Good for maize", "Drought risk" or "Waterlogging risk".
3. **Correct the flawed method from last year.** The previous question asked for cosine similarity using `math.cos()`, which computes the cosine of an *angle*, not the similarity of two vectors. Implement cosine similarity properly with NumPy (dot product ÷ product of norms) and verify it against `scipy.spatial.distance.cosine`.
4. Compute **three** similarity/distance measures between every pair of regions (cosine similarity, Pearson correlation, Euclidean distance) and present them as matrices. Explain why cosine similarity can rate two regions as "similar" even when one is much wetter than the other.
5. Use `scipy.signal.find_peaks` to detect rainy seasons automatically, and classify each region as **unimodal** or **bimodal**. Check that your result matches Uganda's known climate zones.
6. Visualise: (a) all regions' rainfall on one line chart, and (b) a heatmap of crop suitability by month and region.
7. Write a short advisory note (≤ 200 words) for farmers in one region: which crop, and when to plant.

Extension

- Replace the illustrative data with real monthly rainfall (e.g. NASA POWER or UNMA) for at least 10 years, and show year-to-year variability as box plots per month.

Mini-Project 5: Taxi Route Revenue, Pricing & Fleet Planner

Scenario. A taxi (matatu) association runs 14-seater vehicles on three routes out of Kampala. It wants to forecast demand, understand fares and decide how many vehicles to deploy. Daily passenger counts over 10 days (the Ntinda figures are from the original question; the rest are *illustrative*):

```
Kampala-Ntinda = [35, 40, 42, 50, 55, 60, 48, 52, 47, 45] fare UGX 2,000
Kampala-Entebbe = [60, 58, 65, 70, 72, 80, 75, 68, 66, 64] fare UGX 5,000
Kampala-Mukono = [45, 47, 50, 49, 55, 62, 58, 53, 51, 50] fare UGX 3,000
```

Core tasks

1. Create a **Route** class holding passenger counts (NumPy array) and the fare, with methods for daily revenue, total revenue and descriptive statistics (mean, variance, standard deviation via **statistics**).
2. **Supply-demand equilibrium:** for the Ntinda route, assume demand $Q_d = 120 - 0.02P$ and supply $Q_s = 10 + 0.03P$, where P is the fare in UGX and Q is passengers per trip-hour. Write these as a 2×2 linear system and solve it with **scipy.linalg.solve**. Is the current UGX 2,000 fare above or below equilibrium? Interpret the result.
3. Implement a **Forecaster** base class and at least three subclasses: (a) 3-day moving average (the original method), (b) simple exponential smoothing with a tunable α , and (c) linear trend.
4. Evaluate the forecasters with **rolling-origin (walk-forward) backtesting** over days 4-10 and report MAE per model per route. Tune α by grid search.
5. Forecast day 11 revenue for each route with the best model, and plot actual vs forecast revenue.
6. **Fleet planning:** if each vehicle makes 8 one-way trips per day at 14 passengers, how many vehicles should each route deploy on day 11? Add a 15% buffer and round sensibly.

Extension

- Simulate 60 days of data with a weekly pattern (busier on Fridays, quieter on Sundays), and show that a seasonal-naïve forecaster beats the moving average.

Assessment Rubric (applied to each mini-project)

Criterion	Weight	What an excellent submission shows
Correctness & completeness	35%	All core tasks done, results correct and checked against a hand calculation or a second method.
Software design (OOP)	20%	Clear class responsibilities, no duplicated logic, sensible naming, docstrings, type hints, input validation.
Analysis & interpretation	20%	Findings answer the scenario question; assumptions and limitations are stated honestly; methods are critiqued.
Visualisation	10%	Labelled axes with units, titles, legends; the chart supports a specific point made in the text.
Testing & reproducibility	10%	Meaningful pytest tests pass; fixed seeds; <i>Restart & Run All</i> works; clear README.
Extension task	5%	At least one extension attempted with a thoughtful write-up.